



BRANCH AND BOUND

MODULE 2

BCSE204L – DESIGN AND ANALYSIS OF
ALGORITHM



BRANCH AND BOUND

- The **Branch and Bound Technique** is a problem solving strategy, which is most commonly used in optimization problems, where the goal is to minimize a certain value.
- The optimized solution is obtained by means of a state space tree (A state space tree is a tree where the solution is constructed by adding elements one by one, starting from the root. Note that the root contains no element).
- This method is best when used for combinatorial problems with exponential time complexity, since it provides a more efficient solution to such problems.

BRANCH AND BOUND

Algorithm:

- In this technique, the first step is to create a function U (which represents an upper bound to the value that node and its children shall achieve), that we intend to minimize.
- We call this function the objective function. Note that the branch and bound technique can also be used for maximization problems, since multiplying the objective function by -1 converts the problem to a minimization problem.
- Let this function have an initial maximum value, according to the conditions of the given problem.
- Also, let U_0 be the initial value of U . We also calculate a cost function C which will give us the exact value that the particular node shall achieve

BRANCH AND BOUND

Algorithm:

- The next question is the order in which the tree is to be searched.
- For this, there exist multiple types of branch and bound, which we shall discuss in detail later.
- For now, let us assume that there is a set S consisting of subsets of the given set of values in the order in which they need to be searched.

BRANCH AND BOUND

Algorithm:

- The algorithm of the branch and bound method for this problem will be as follows:
- For each subset s in S , do the following:
 - Calculate the value of $U(s)$.
 - If $U(s)$ and U_0 , then $U_0 = U(s)$.
- In the end, the subset s for which the current value of U_0 is obtained will be the best solution to the given problem, and the value of the cost function at that node will give us the solution.
- Here, after each level, the value of U_0 tells us that there shall be a node with cost less than that value.

BRANCH AND BOUND

Types of Solution:

- For a branch and bound problem, there are two ways in which the solution can be represented:
- **Variable size solution:** This solution provides the subset of the given set that gives the optimized solution for the given problem. For example, if we are to select a combination of elements from {A, B, C, D, E} that optimizes the given problem, and it is found that A, B, and E together give the best solution, then the solution will be {A, B, E}.
- **Fixed size solution:** This solution is a sequence of 0s and 1s, with the digit at the i^{th} position denoting whether the i^{th} element should be included or not. Hence, for the earlier example, the solution will be given by {1, 1, 0, 0, 1}.

BRANCH AND BOUND

Types of Branch and Bound:

- There are multiple types of the Branch and Bound method, based on the order in which the state space tree is to be searched.
- We shall now discuss each of these methods in detail. We will be using the variable solution method to denote the solutions in these methods.
 - **FIFO Branch and Bound**
 - **LIFO Branch and Bound**
 - **Least Cost Branch and Bound**

BRANCH AND BOUND

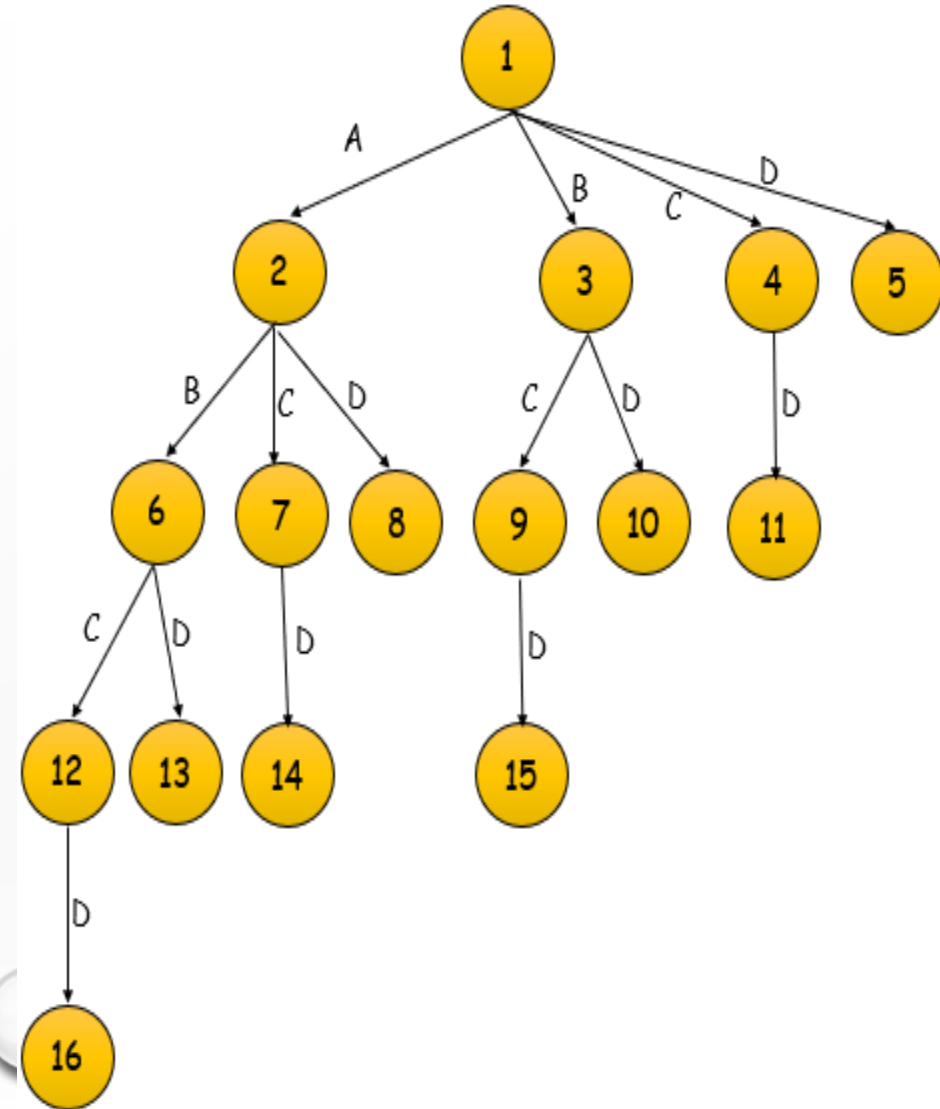
FIFO Branch and Bound

- The First-In-First-Out approach to the branch and bound problem follows the queue approach in creating the state-space tree.
- Here, **breadth first search** is performed, i.e., the elements at a particular level are all searched, and then the elements of the next level are searched, starting from the first child of the first node in the previous level.

BRANCH AND BOUND

FIFO Branch and Bound

- For a given set $\{A, B, C, D\}$, the state space tree will be constructed as follows :
- Here, note that the number assigned to the node signifies the order in which the tree shall be constructed.
- The element next to the set denotes the next element to be added to the subset. Note that if an element is getting added, it is assumed here that all elements in the set preceding that element are not added.
- For example, in node 4, D is getting added. This implies that elements A, B and C are not added.



BRANCH AND BOUND

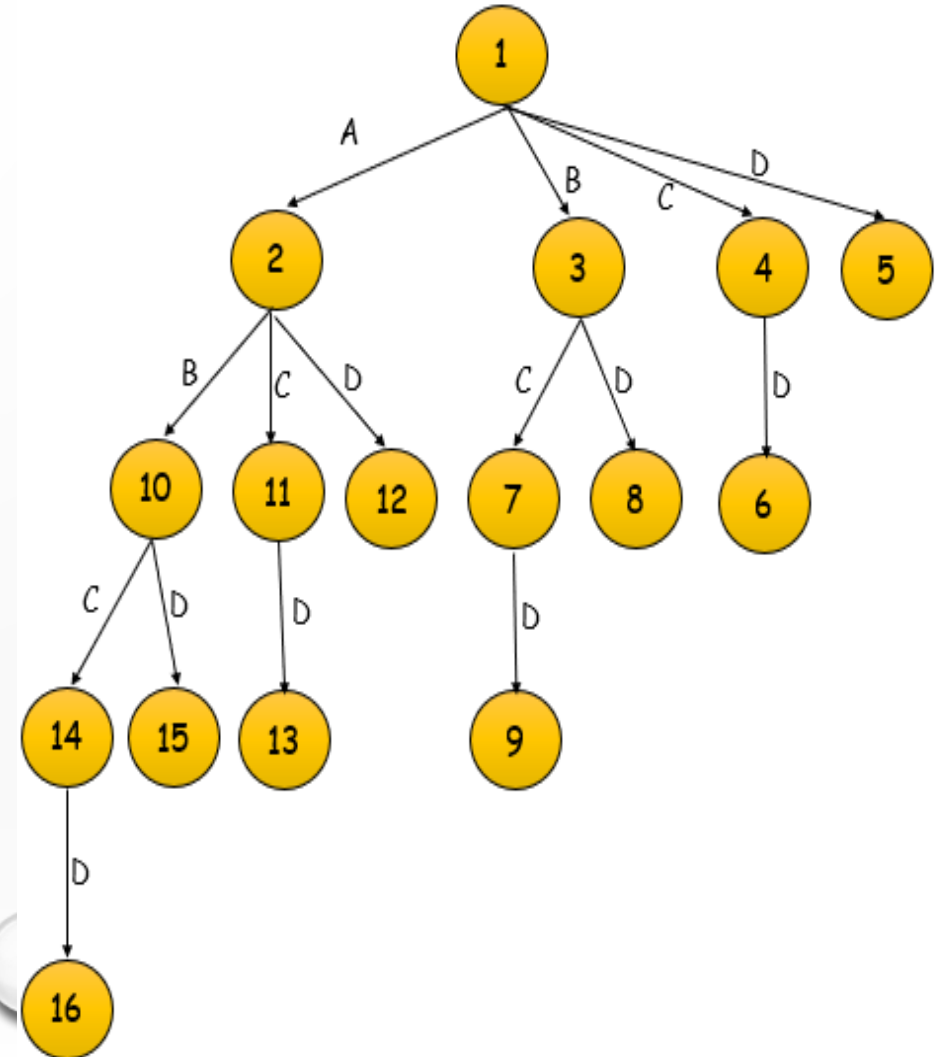
LIFO Branch and Bound

- The Last-In-First-Out approach to this problem follows the stack approach in creating the state space tree.
- Here, when nodes get added to the state space tree, think of them as getting added to a stack.
- When all nodes of a level are added, we pop the topmost element from the stack and then explore it.

BRANCH AND BOUND

FIFO Branch and Bound

- For a given set $\{A, B, C, D\}$, the state space tree will be constructed as follows :
- Here, one can see that the main difference lies in the order in which the nodes have been explored.



BRANCH AND BOUND

Least Cost Branch and Bound

- This method of exploration uses the cost function in order to explore the state space tree.
- Although the previous two methods calculate the cost function at each node, this is not used as a criterion for further exploration.
- In this method, after the children of a particular node have been explored, the next node to be explored would be that node out of the unexplored nodes which has the least cost.
- For example, in the previous example, after reaching node 5, the next node to be explored would be that which has the least cost among nodes 2, 3, 4, 5.

BRANCH AND BOUND

Why use Branch and Bound?

- The Branch and Bound method is preferred over other similar methods such as backtracking when it comes to optimization problems.
- Here, the cost and the objective function help in finding branches that need not be explored.
- Suppose the cost of a particular node has been determined. If this value is greater than that of U_0 , this means that there is no way this node or its children shall give a solution.
- Hence, we can kill this node and not explore its further branches. This method helps us rule out cases not worth exploring, and is therefore more efficient.

BRANCH AND BOUND

Application or Problem that can be solved using BB:

- **Job Sequencing:** Suppose there is a set of N jobs and a set of N workers. Each worker takes a specific time to complete each of the N jobs. The job sequencing problem deals with finding that order of the workers, which minimizes the time taken to complete the job.
- **0/1 Knapsack problem:** Given a set of weights of N objects and a sack which can carry a maximum of W units. The problem deals with finding that subset of items such that the maximum possible weight is carried in the sack. Here, one cannot take part of an object, i.e., one can either take an object or not take it. This problem can also be solved using the backtracking, brute force and the dynamic programming approach.
- **Traveling Salesman Problem:** Here, we are given a set of N cities, and the cost of traveling between all pairs of cities. The problem is to find a path such that one starts from a given node, visits all cities exactly once, and returns back to the starting city.

BRANCH AND BOUND

Dynamic Programming Vs Branch Bound:

Dynamic Programming

Constructs the solution in form of a table.

Solves all possible instances of problem of size n .

Does not require a bounding function.

After constructing the table, it needs to be traced back to find the solution sequence.

Applications: Binomial Coefficient, Making a change, Knapsack problem, Multistage graph, optimal Binary Search Tree, Matrix Chain Multiplication, Longest Common Subsequence, Bellman-Ford (SSSP) Algorithm, Assembly Line Scheduling, Traveling Salesman Problem, Flow Shop Scheduling,

Branch and Bound

Constructs the solution in form of a tree.

Only solves promising instances from the set of instances at any given point.

Needs to compute and apply a bounding function at each node.

The solution sequence is implicit, a leaf node of the tree is the final solution.

Applications:

Traveling salesman problem
Knapsack problem
Job Sequencing Problem

BRANCH AND BOUND

Backtracking Vs Branch Bound:

Backtracking

Backtracking is normally used to solve decision problems

Nodes in the state-space tree are explored in depth-first order in the backtracking method

It realizes that it has made a bad choice & undoes the last choice by backing up.

Branch and Bound

Branch and bound is used to solve optimization problems

Nodes in the tree may be explored in depth-first or breadth-first order in branch and bound method

It realizes that it already has a better optimal solution that the pre-solution leads to so it abandons that pre-solution.

BRANCH AND BOUND

Backtracking Vs Branch Bound:

Backtracking

The feasibility function is used in backtracking.

The next move from the current state can lead to a bad choice

On successful search of a solution in state-space tree, the search stops

Backtracking is more efficient.

Branch and Bound

Branch-and-Bound involves a bounding function.

The next move is always towards a better solution

The entire state space tree is searched in order to find the optimal solution

Branch-and-Bound is less efficient.

BRANCH AND BOUND

Backtracking Vs Branch Bound:

Backtracking

Applications:

N Queen Problem

Knapsack Problem

Sum of subsets problem

Hamiltonian cycle problem, Graph coloring problem

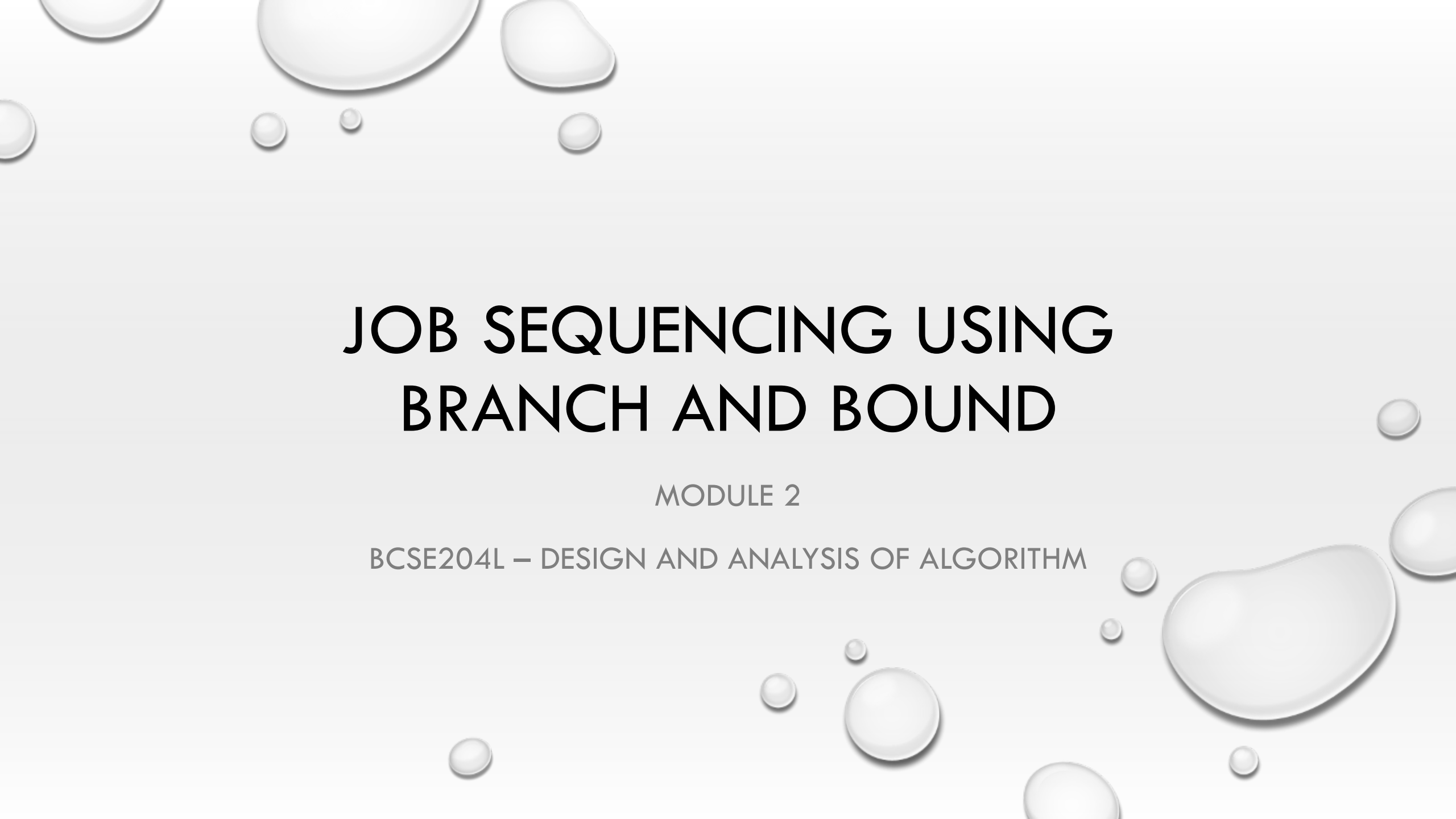
Branch and Bound

Applications:

Traveling salesman problem

Knapsack problem

Job sequencing problem

The background of the slide is decorated with numerous realistic water droplets of various sizes. Some are large and prominent, while others are small and subtle. They are scattered across the slide, with a higher concentration in the top-left and bottom-right corners, creating a clean, modern, and aesthetically pleasing look.

JOB SEQUENCING USING BRANCH AND BOUND

MODULE 2

BCSE204L – DESIGN AND ANALYSIS OF ALGORITHM

JOB SEQUENCING USING BRANCH AND BOUND

- Job Sequencing using Branch and Bound: Given n jobs with profit, execution time, and deadline, achieve the schedule which maximizes the profit.
- We will solve the problem using the FIFO branch and bound with the variable tuple and fixed tuple representations.
- Each job i is represented by the tuple (P_i, d_i, t_i) , where P_i , d_i , and t_i represent profit, deadline, and execution time associated with job i .
- If job i is completed on or before its deadline, profit P_i is earned. But if the job i finish after its deadline, then a penalty P_i will incur.
- The brute force method finds 2^n schedules for n jobs and finds the best from them. Branch and bound is a more efficient way of finding the optimal schedule.

JOB SEQUENCING USING BRANCH AND BOUND

- Example: Select optimal subset J with an optimal penalty for the following data. What will be the penalty corresponding to the optimal solution?

- We can use either one
 - Variable length solution
 - Fixed length solution

Job	P_i	d_i	t_i
1	5	1	1
2	10	3	2
3	6	2	1
4	3	1	1

JOB SEQUENCING USING BRANCH AND BOUND

- Branch and Bound can be used only for minimization problems.
 - Hence we convert maximization problem to minimization problem.
 - We convert profit to penalty, so minimum penalty means maximum profit.
 - Calculate **cost function** for node x in state space tree

$$\hat{C} = \sum_{i \notin s_k} P_i$$

- S_k is the subset of jobs selected at node k.
- Calculate **upper bound**

$$u = \sum_{i \in s} P_i$$

Job	P_i	d_i	t_i
1	5	1	1
2	10	3	2
3	6	2	1
4	3	1	1

JOB SEQUENCING USING BRANCH AND BOUND

$$\hat{C} = \sum_{i \notin s_k} P_i$$

$$u = \sum_{i \in s} P_i$$

Job	P_i	d_i	t_i
1	5	1	1
2	10	3	2
3	6	2	1
4	3	1	1

- $\hat{C}$ is sum of penalties till the k job considered.
- u is sum of all penalties except that included in solution s.

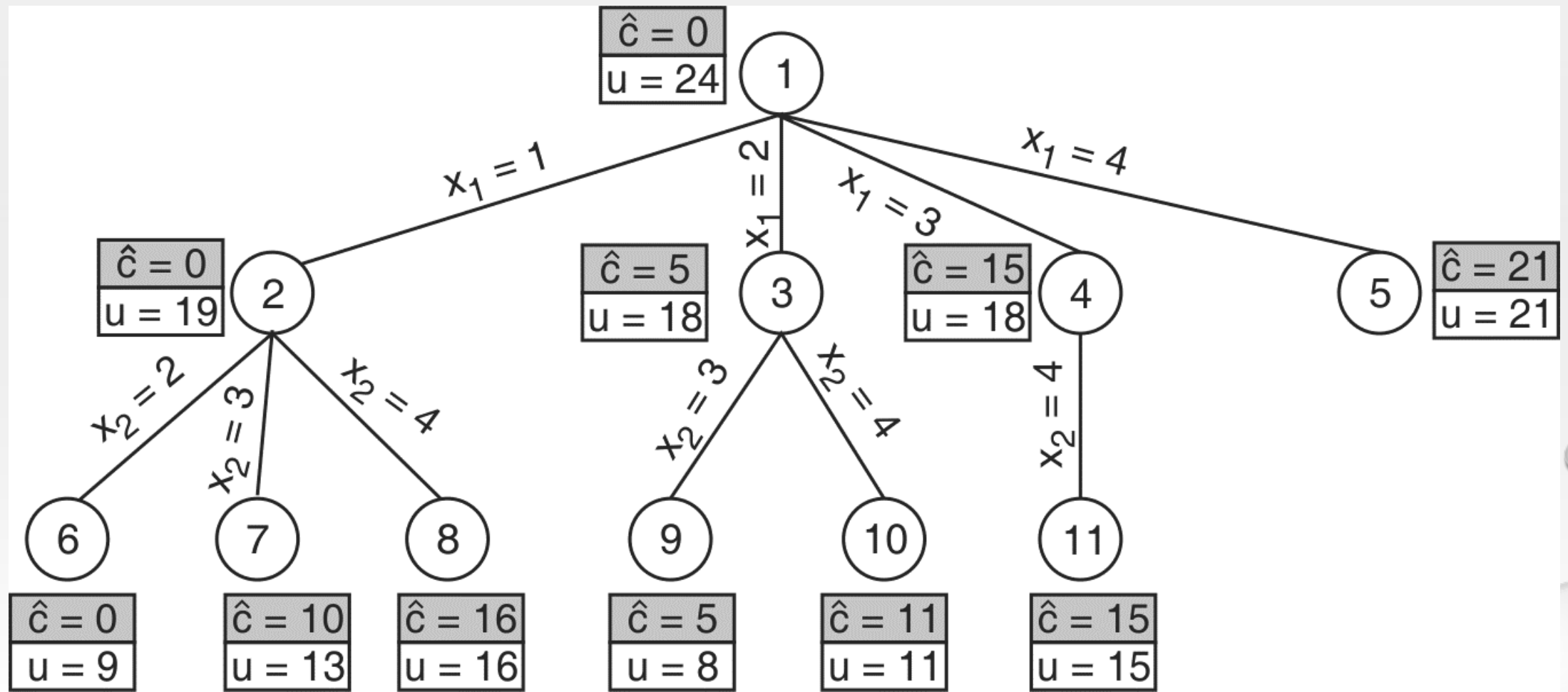
JOB SEQUENCING USING BRANCH AND BOUND

Algorithm

1. Let initially $upper = \infty$
2. If $\hat{C}$ is less than $upper$ and u is less than $upper$ then update $upper$ by u .
3. If $\hat{C}$ is greater than $upper$ then kill the node
4. ***When we are adding a new job we have to check the total time of included job is less than max deadline. If greater then it is infeasible.***

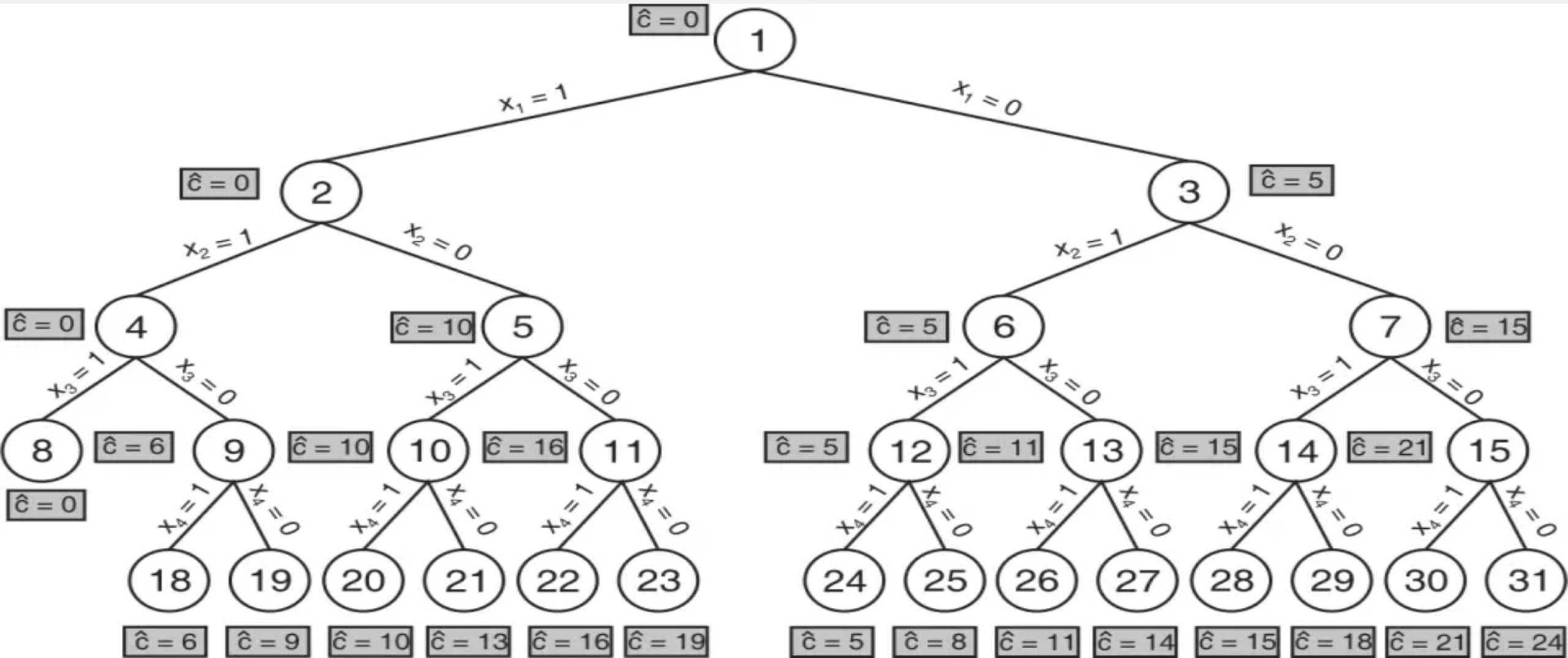
JOB SEQUENCING USING BRANCH AND BOUND

State-space tree for variable tuple formulation



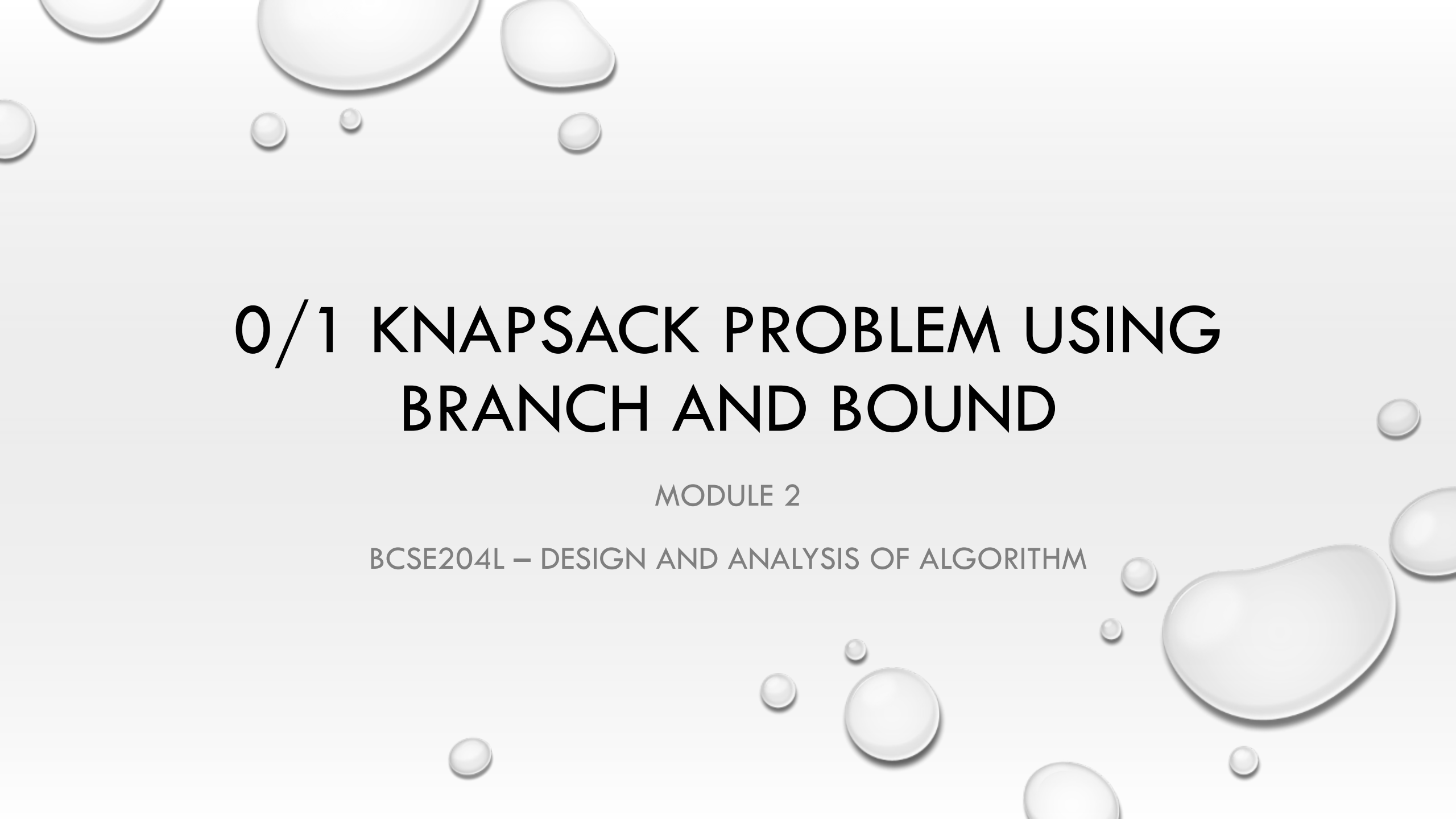
JOB SEQUENCING USING BRANCH AND BOUND

State-space tree for fixed tuple formulation



JOB SEQUENCING USING BRANCH AND BOUND

- Final Solution using Variable Length and Fixed Length Formulation
- Minimum cost $\hat{C} = 5$
- Minimum penalty $u = 8$
- Job selected are J2 and J3 saving maximum profit with $10+6 = 16$ and saving penalty of $5+3 = 8$
- Final answer is $\mathbf{S} = \{\mathbf{J2}, \mathbf{J3}\}$ for Variable length and $\mathbf{S} = \{\mathbf{0}, \mathbf{1}, \mathbf{1}, \mathbf{0}\}$ for Fixed length formulation.

The background of the slide is decorated with numerous realistic water droplets of various sizes. Some are large and prominent, while others are small and scattered. They are rendered with soft shadows and highlights, giving them a three-dimensional appearance. The droplets are distributed across the entire slide, with a higher concentration in the top and bottom corners.

0/1 KNAPSACK PROBLEM USING BRANCH AND BOUND

MODULE 2

BCSE204L – DESIGN AND ANALYSIS OF ALGORITHM

0/1 KNAPSACK PROBLEM USING BB

Introduction

- We are given a set of n objects which each have a value v_i and a weight w_i .
- The objective of the 0/1 Knapsack problem is to find a subset of objects such that the total value is maximized, and the sum of weights of the objects does not exceed a given threshold W .
- An important condition here is that one can either take the entire object or leave it. It is not possible to take a fraction of the object.
- Example where $n = 4$, and the values are given by $\{10, 12, 12, 18\}$ and the weights given by $\{2, 4, 6, 9\}$.
- The maximum weight is given by $W = 15$. Here, the solution to the problem will be including the first, third and the fourth objects.

0/1 KNAPSACK PROBLEM USING BB

Introduction

- We are given a set of n objects which each have a value v_i and a weight w_i .
- The objective of the 0/1 Knapsack problem is to find a subset of objects such that the total value is maximized, and the sum of weights of the objects does not exceed a given threshold W .
- An important condition here is that one can either take the entire object or leave it. It is not possible to take a fraction of the object.
- Example where $n = 4$, and the values are given by $\{10, 12, 12, 18\}$ and the weights given by $\{2, 4, 6, 9\}$.
- The maximum weight is given by $W = 15$. Here, the solution to the problem will be including the first, third and the fourth objects.

0/1 KNAPSACK PROBLEM USING BB

Why branch and bound?

- **Greedy approach** works only for **Fractional knapsack problem**.
- If weights are not integers, **dynamic programming** will not work.
- There are 2^n possible combinations of item, complexity for **brute force** goes exponentially.

0/1 KNAPSACK PROBLEM USING BB

Why branch and bound?

- **Branch and Bound** is an algorithm design paradigm which is generally used for solving **combinatorial optimization problems**.
- These problems typically exponential in terms of time complexity and may require exploring all possible permutations in worst case.
- **BB solve these problems relatively quickly.**
- **Combinatorial optimization problem** is an optimization problem, where an optimal solution has to be identified from a finite set of solutions.

0/1 KNAPSACK PROBLEM USING BB

Knapsack Problem using Branch and Bound

- As discussed earlier, the goal of knapsack problem is to maximize $\sum_i p_i x_i$ given the constraints $\sum_i w_i \leq M$, where M is the size of the knapsack.
- A maximization problem can be converted to a minimization problem by negating the value of the objective function.
- The modified knapsack problem is stated as,

**Minimize - $\sum_i p_i x_i$ subjected to $\sum_i w_i \leq M$,
where $x_i = \{0, 1\}, 1 \leq i \leq n$.**

0/1 KNAPSACK PROBLEM USING BB

Knapsack Problem using Branch and Bound

- Node satisfying the constraint $\sum_i w_i \leq M$ in state space tree is called the **answer state**, the remaining nodes are infeasible.
- For the minimum cost answer node, we need to define $\hat{c} = - \sum_i p_i x_i$ for every answer node x_i .
- Cost for infeasible leaf node would be ∞ .
- For all non-leaf nodes, cost function is recursively defined as
$$c(x) = \min\{c(Lchild(x)), c(Rchild(x))\}$$
- For every node x , $\hat{c} \leq c(x) \leq u(x)$
- Algorithm for knapsack problem using branch and bound is described below :
- For any node N, upper bound for feasible left child remains N. But upper bound for its right child needs to be calculated.

0/1 KNAPSACK PROBLEM USING BB

LC – BB for Knapsack Problem

- LC branch and bound solution for knapsack problem is derived as follows :
 1. Derive state space tree.
 2. Compute lower bound $\hat{C}$ and upper bound $\hat{C}$ for each node in state space tree.
 3. **If lower bound is greater than upper bound then kill that node.**
 4. Else **select node with minimum lower bound as E-node.**
 5. Repeat step 3 and 4 until all nodes are examined.
 6. The node with minimum lower bound value $\hat{C}(.)$ is the answer node. Trace the path from leaf to root in the backward direction to find the solution tuple.

0/1 KNAPSACK PROBLEM USING BB

LC branch and bound solution for knapsack:

- The bounding function is a heuristic computation. For the same problem, there may be different bounding functions.
- Apart from the above-discussed bounding function, another very popular bounding function for knapsack is,
- **Cost Function** : Profit of selected items with fractional ,

$$\hat{C} = -u - \left(M - \sum_{i \in S} w_i \right) * \left(\frac{p_{i+1}}{w_{i+1}} \right)$$

- **Upper Bound function**: Profit of selected items without fractional

$$u = - \sum_{1 \leq i \leq n} p_i, \text{ such that } \sum_{1 \leq i \leq n} w_i \leq M$$

- where,
 - S selected item that satisfying the constraint $\sum_i w_i \leq M$
 - $i + 1$ is value/profit associated with non-selected items from the first i items.
 - M is the capacity of the knapsack.
 - w is the weight of selected items from first i items Derive state space tree.

0/1 KNAPSACK PROBLEM USING LC - BB

Example: Solve the following instance of knapsack using LC-BB for knapsack capacity $M = 15$. LC-branch and bound solution for knapsack:.

i	P_i	W_i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING LC - BB

Solution:

- Let us compute $u(1)$ and $\hat{C}(1)$.
- If we include first three item then $\sum_i w_i \leq M$, but if we include 4th item, it exceed knapsack capacity M , so

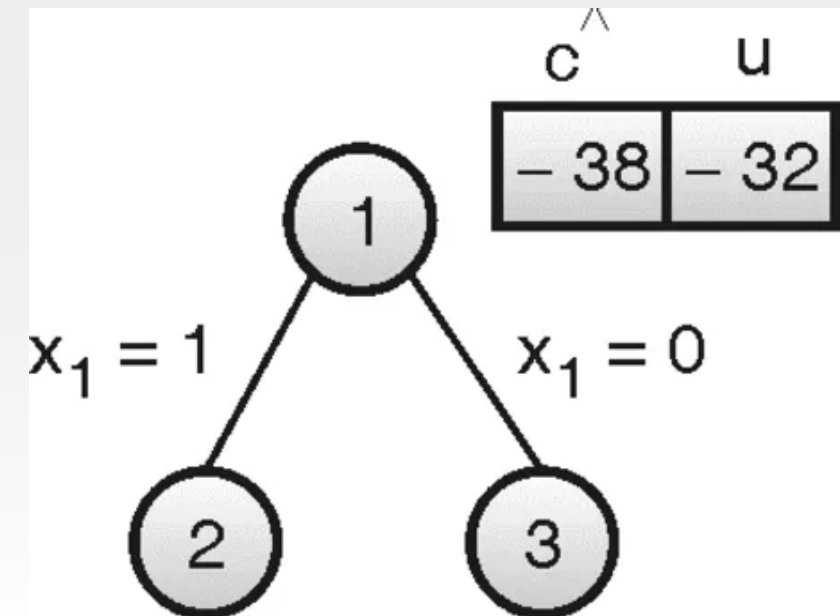
i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

- $u(1) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$= -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(1) = u(1) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{W_{i+1}}\right)$

$$= -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$



0/1 KNAPSACK PROBLEM USING LC - BB

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

Node 2: For $x_1 = 1$ – Item 1 selected for Node 1

- Item 1 is include So value of $u(2)$ and $\hat{C}(2)$.

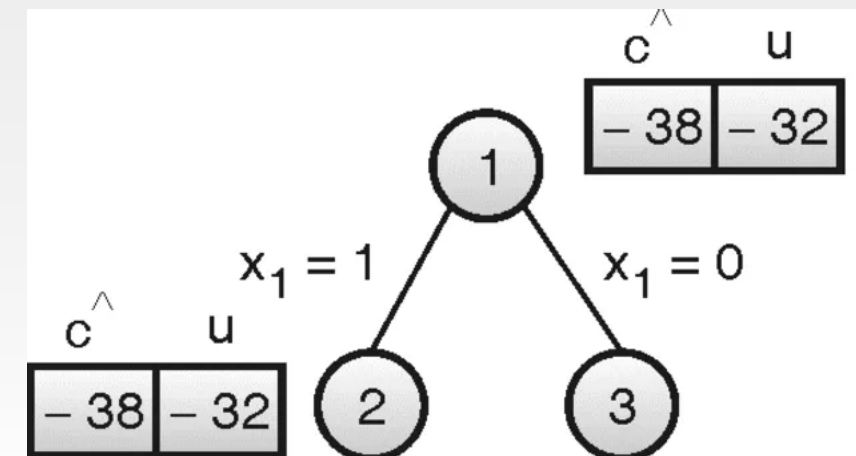
- $u(2) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(2) = -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(2) = u(2) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(2) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

Node 2 is inserted in list of live nodes.



0/1 KNAPSACK PROBLEM USING LC - BB

Node 3: For $x_1 = 0$ – Item 1 not selected for Node 1

- We are excluding item 1 and including 2 and 3. Item 4 cannot be accommodated in knapsack along with items 2 and 3. So value of $u(3)$ and $\hat{C}(3)$.

- $u(3) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(3) = -(P_2 + P_3) = -(10 + 12) = -22$$

- $\hat{C}(3) = u(3) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

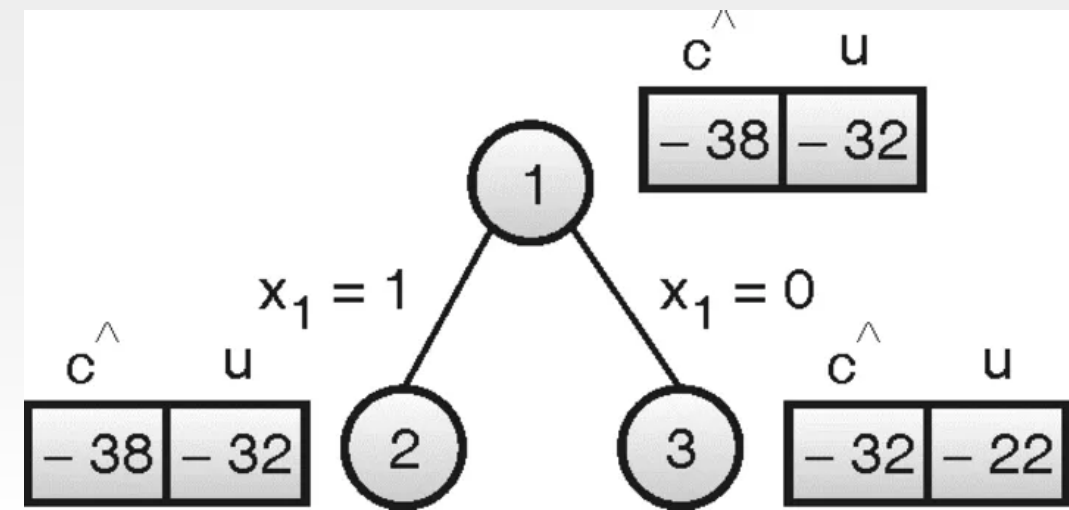
$$\hat{C}(3) = -22 - (15 - (4 + 6)) * \left(\frac{18}{9}\right) = -32$$

Node 3 is inserted in the list of live nodes.

At level 1, node 2 has minimum $\hat{C}$,

so it becomes E-node.

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING LC - BB

Node 4: For $x_2 = 1$ – Item 2 selected for Node 2

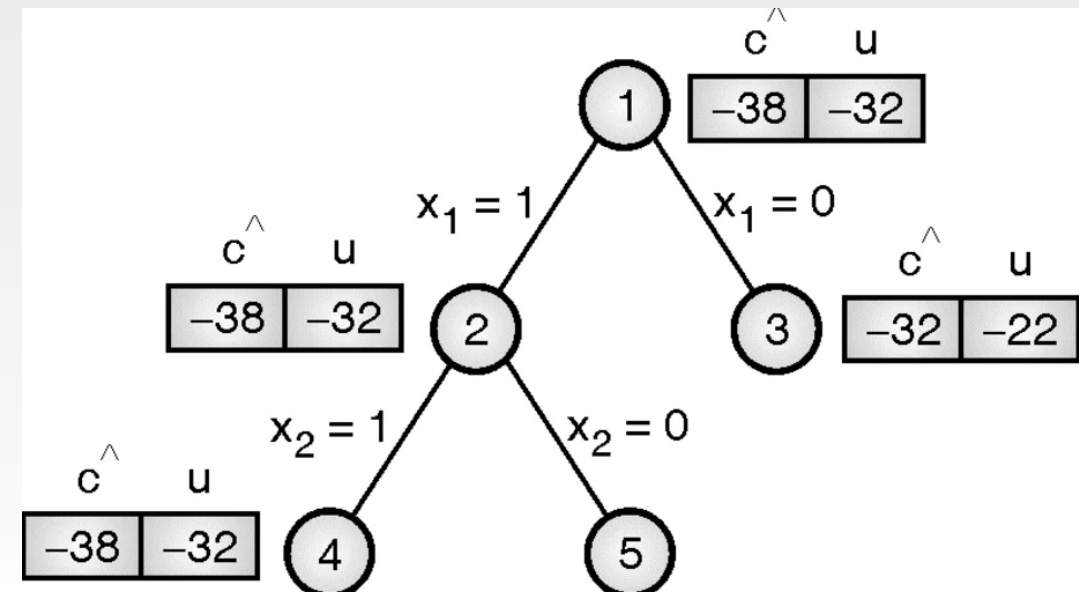
- Item 1 is already added at node 2, and we must have to include item 2 at this node. After adding items 1 and 2, the knapsack can accommodate item 3 but not 4. So value of $u(4)$ and $\hat{C}(4)$.
- $u(4) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(4) = -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(4) = u(4) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(4) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING LC - BB

Node 5: For $x_2 = 0$ – Item 2 not selected for Node 2

- At node 5, item 1 is already added, we must have to skip item 2. Only item 3 can be accommodated in knapsack after inserting item 1. So value of $u(5)$ and $\hat{C}(5)$.
- $u(5) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

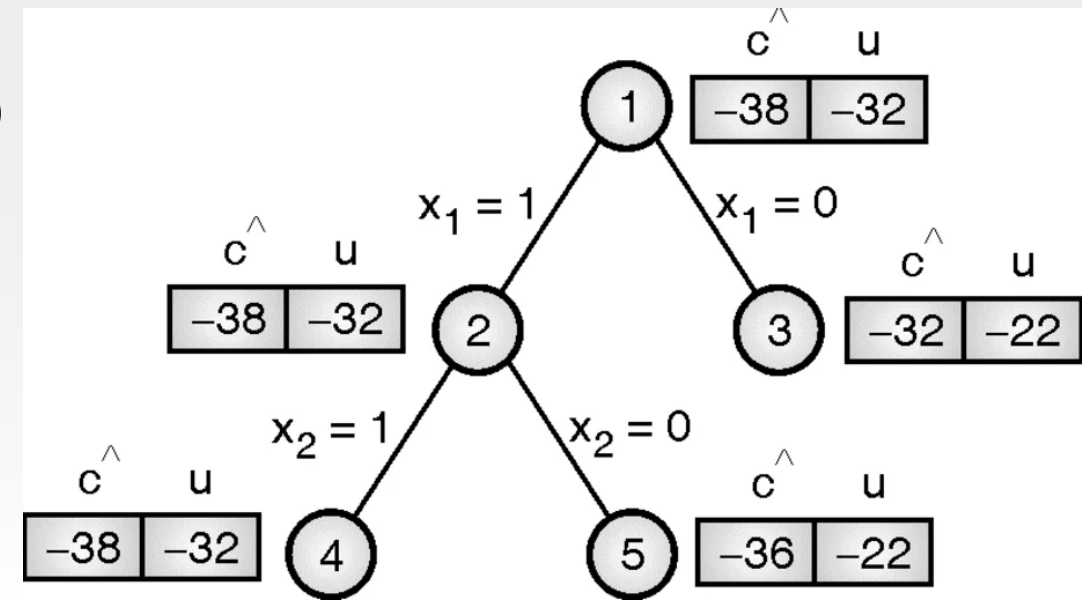
$$u(5) = -(P_1 + P_3) = -(10 + 12) = -22$$

- $\hat{C}(5) = u(5) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(5) = -22 - (15 - (2 + 6)) * \left(\frac{18}{9}\right) = -36$$

**At level 2, node 4 has minimum $\hat{C}$,
so it becomes E-node.**

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING LC - BB

Node 6: For $x_3 = 1$ – Item 3 selected for Node 4

- At node 4, item 1 and 2 are already added, we have to add item 3. After including item 1, 2 and 3, item 4 cannot be accommodated. So value of $u(6)$ and $\hat{C}(6)$.

- $u(6) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(6) = -(10 + 10 + 12) = -32$$

- $\hat{C}(6) = u(6) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(6) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING LC - BB

Node 7: For $x_3 = 0$ – Item 3 not selected for Node 4

- On excluding item 3, we can accommodate item 1, 2 and 4 in knapsack. So value of $u(5)$ and $\hat{C}(5)$.

- $u(7) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(7) = -(10 + 10 + 18) = -38$$

- $\hat{C}(7) = u(7) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

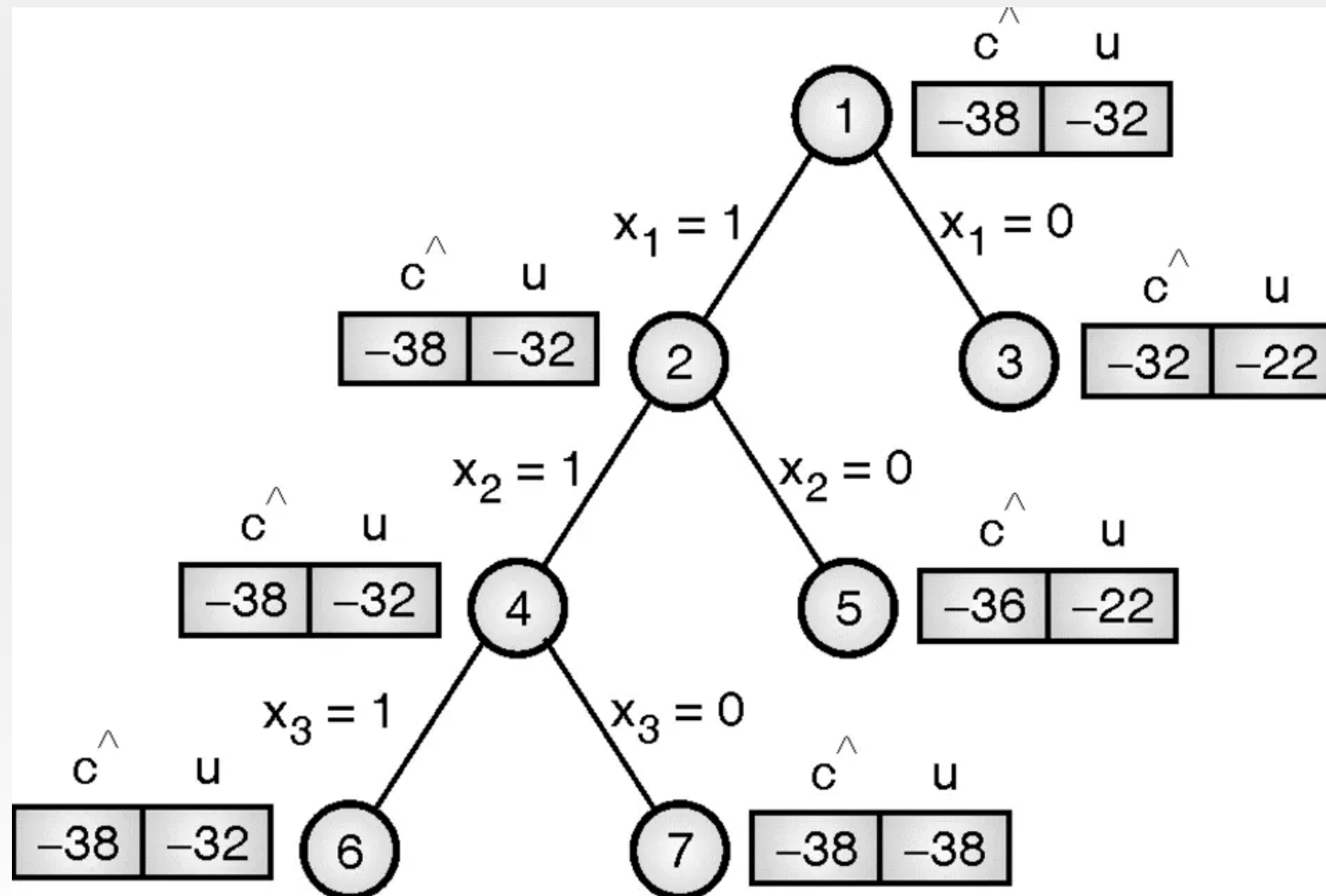
$$\hat{C}(7) = -38 - (15 - (2 + 4 + 9)) * \left(\frac{12}{6}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

When there is tie, we can select any node. Let's make node 7 E-node.

0/1 KNAPSACK PROBLEM USING LC - BB

State Space Tree:



i	P_i	W_i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING LC - BB

Node 8: For $x_4 = 1$ – Item 4 selected for Node 7

- So value of $u(8)$ and $\hat{C}(8)$.
- $u(8) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(8) = -(10 + 10 + 18) = -38$$

- $\hat{C}(8) = u(8) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(8) = -38 - (15 - (2 + 4 + 9)) * \left(\frac{12}{6}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING LC - BB

Node 9: For $x_3 = 0$ – Item 3 not selected for Node 7

- This node excludes both the items, 3 and 4. We can add only item 1 and 2. So value of $u(9)$ and $\hat{C}(9)$.

- $u(9) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(9) = -(10 + 10) = -20$$

- $\hat{C}(9) = u(9) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(9) = -20 - (15 - (2 + 4)) * \left(\frac{0}{0}\right) = -20$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

At last level, node 8 has minimum $\hat{C}$, so node 8 would be the answer node. By tracing from node 8 to root, we get item 4, 2 and 1 in the knapsack.

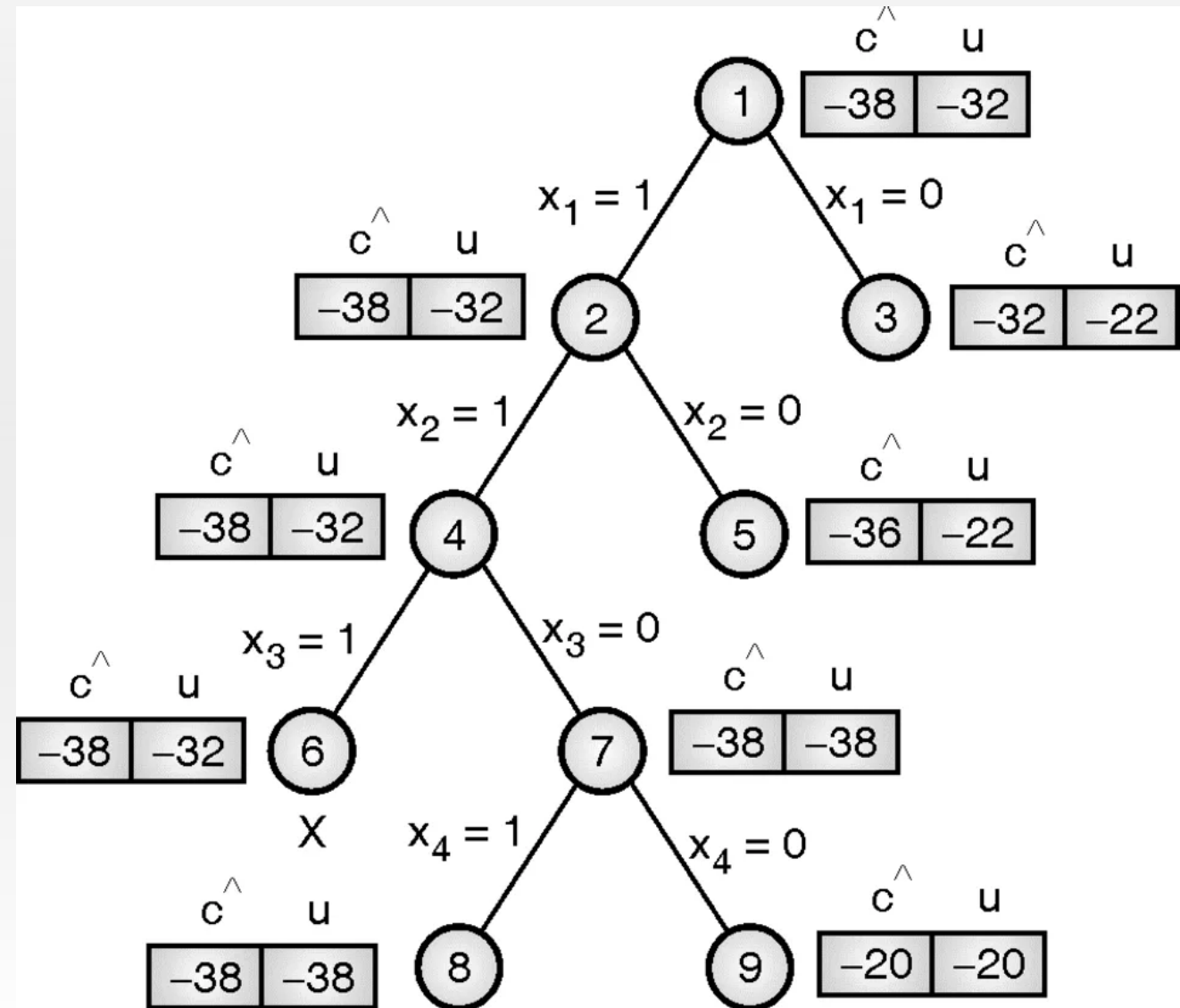
0/1 KNAPSACK PROBLEM USING LC - BB

State Space Tree:

Solution Vector $X = \{x_1, x_2, x_4\}$

Profit = $P_1 + P_2 + P_4 = 38$

i	P_i	W_i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

FIFO branch and bound solution for knapsack:

- In FIFO branch and bound approach, variable tuple size state space tree is drawn.
- For each node N , cost function $\hat{C}(\cdot)$ and upper bound $u(\cdot)$ is computed similarly to the previous approach.
- In LC search, **E node is selected from two child of current node.**
- In FIFO branch and bound approach, both the children of siblings are inserted in list and **most promising node is selected as new E node.**
- Let us consider the same example :

0/1 KNAPSACK PROBLEM USING FIFO - BB

Example: Solve the following instance of knapsack using FIFO-BB for knapsack capacity $M = 15$. LC branch and bound solution for knapsack:.

i	P_i	W_i
1	10	2
2	10	4
3	12	6
4	18	9

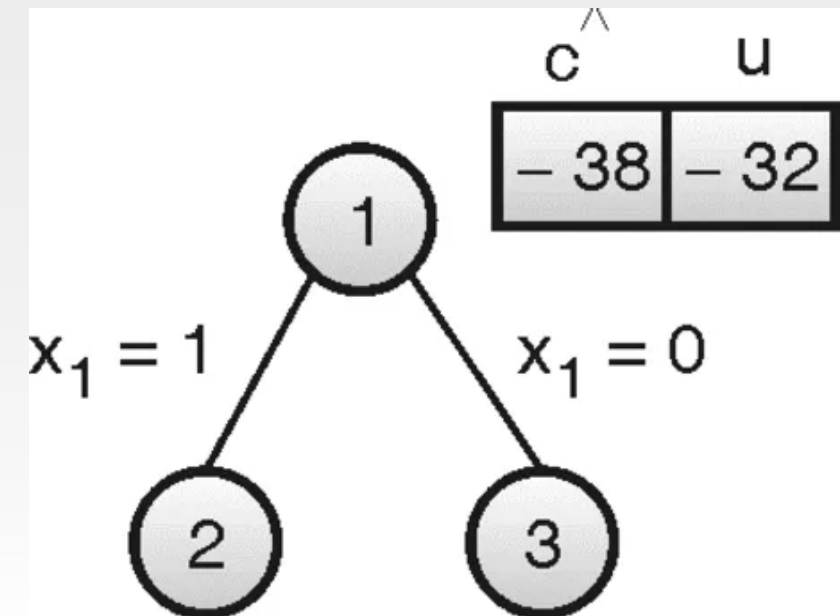
0/1 KNAPSACK PROBLEM USING FIFO - BB

Solution:

- Let us compute $u(1)$ and $\hat{C}(1)$.
- $u(1) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,
$$= -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$
- $\hat{C}(1) = u(1) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$
$$= -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

Upper = $\hat{C}(1) = 38$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 2: For $x_1 = 1$ – Item 1 selected for Node 1

- Item 1 is include So value of $u(2)$ and $\hat{C}(2)$.

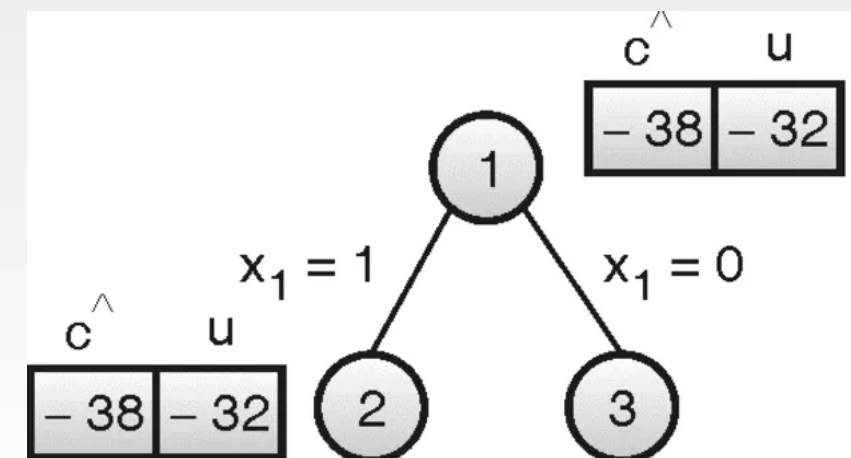
- $u(2) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(2) = -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(2) = u(2) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(2) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 3: For $x_1 = 0$ – Item 1 not selected for Node 1

- We are excluding item 1 and including 2 and 3. Item 4 cannot be accommodated in knapsack along with items 2 and 3. So value of $u(3)$ and $\hat{C}(3)$.

- $u(3) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(3) = -(P_2 + P_3) = -(10 + 12) = -22$$

- $\hat{C}(3) = u(3) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

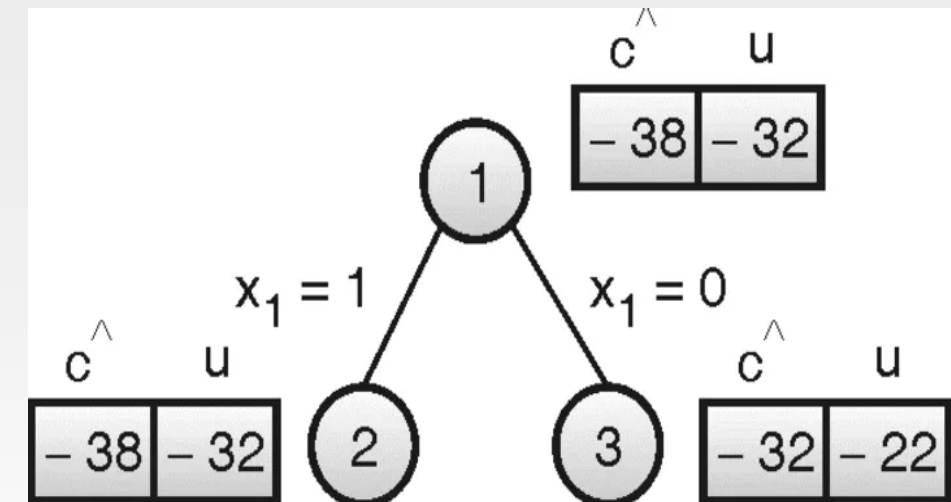
$$\hat{C}(3) = -22 - (15 - (4 + 6)) * \left(\frac{18}{9}\right) = -32$$

Upper = 38

In LC Approach, node 2 would be as E-node as it has minimum value.

But in FIFO approach, all child of node 2 and 3 are expanded and the most promising child becomes E-node.

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 4: For $x_2 = 1$ – Item 2 selected for Node 2

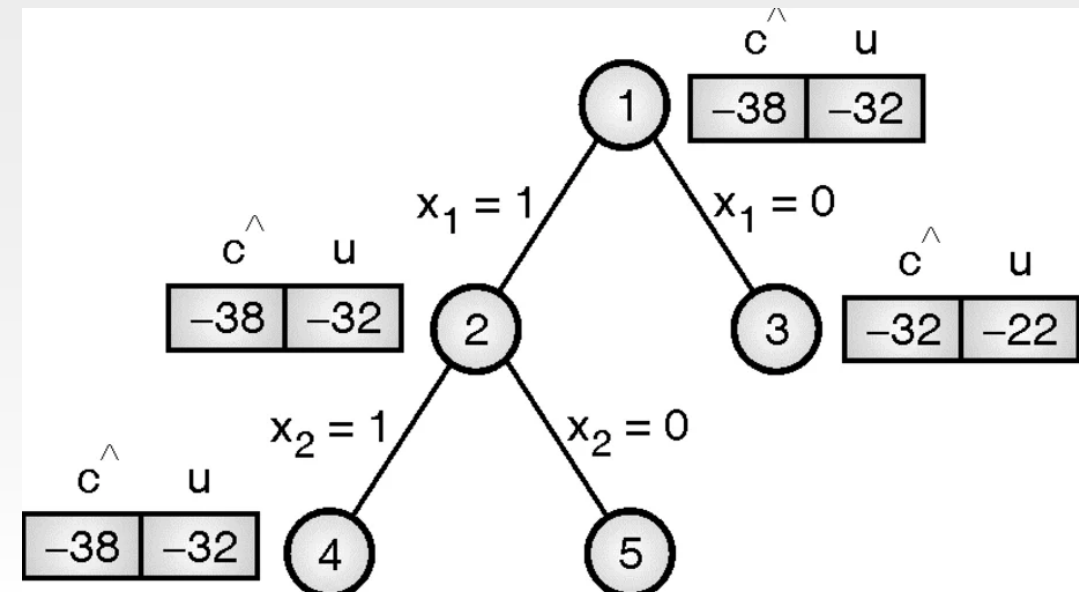
- Value of $u(4)$ and $\hat{C}(4)$.
- $u(4) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(4) = -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(4) = u(4) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(4) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 5: For $x_2 = 0$ – Item 2 not selected for Node 2

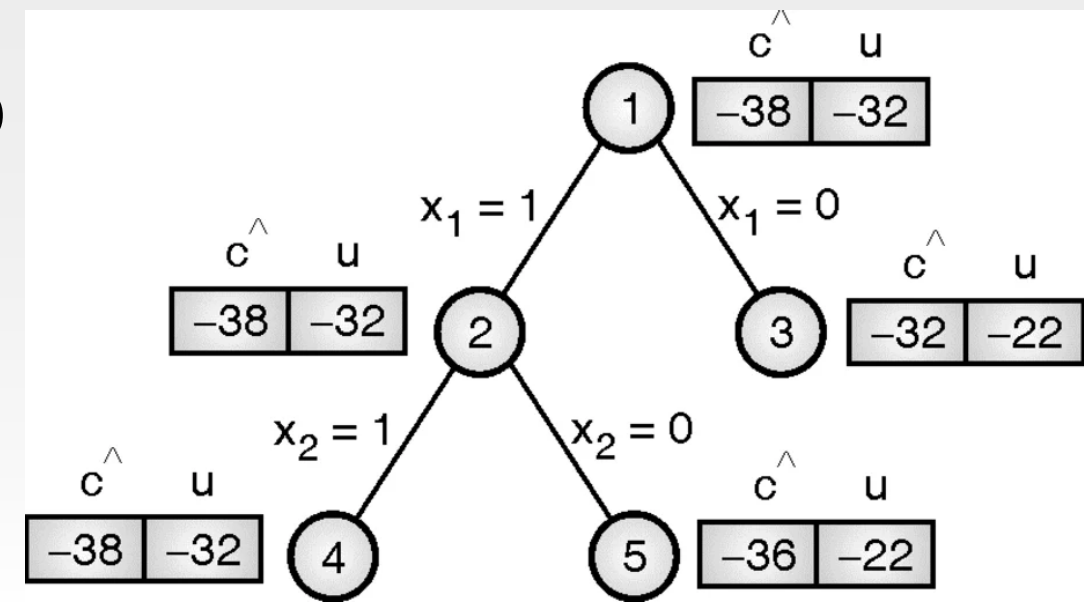
- We are excluding item 1, including 2 and 3. Item 4 cannot be accommodated in a knapsack along with 2 and 3. So value of $u(5)$ and $\hat{C}(5)$.
- $u(5) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(5) = -(P_1 + P_3) = -(10 + 12) = -22$$

- $\hat{C}(5) = u(5) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(5) = -22 - (15 - (2 + 6)) * \left(\frac{18}{9}\right) = -36$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9



0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 6: For $x_2 = 1$ – Item 2 selected for Node 3

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

- Value of $u(6)$ and $\hat{C}(6)$.
- $u(6) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(6) = -(P_2 + P_3) = -(10 + 12) = -22$$

- $\hat{C}(6) = u(6) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{W_{i+1}}\right)$

$$\hat{C}(6) = -22 - (15 - (4 + 6)) * \left(\frac{18}{9}\right) = -32$$

0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 7: For $x_2 = 0$ – Item 2 not selected for Node 3

- Value of $u(7)$ and $\hat{C}(7)$.
- $u(7) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,
$$u(7) = -(12 + 18) = -30$$
- $\hat{C}(7) = u(7) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(7) = -30 - (15 - (6 + 9)) * \left(\frac{0}{0}\right) = -30$$

Out of node 4, 5, 6 and 7, $\hat{C} > \text{upper}$ are killed, so kill node 5,6,7.

Remaining Node 4 is live and added in list. node 4 has minimum $\hat{C}$ value, so it becomes next E-node.

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 8: For $x_3 = 1$ – Item 3 selected for Node 4

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

- Value of $u(8)$ and $\hat{C}(8)$.
- $u(8) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(8) = -(P_1 + P_2 + P_3) = -(10 + 10 + 12) = -32$$

- $\hat{C}(8) = u(8) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{W_{i+1}}\right)$

$$\hat{C}(8) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{18}{9}\right) = -38$$

0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 9: For $x_3 = 0$ – Item 3 not selected for Node 4

- Value of $u(9)$ and $\hat{C}(9)$.
- $u(9) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(9) = -(P_1 + P_2 + P_3) = -(10 + 10 + 18) = -38$$

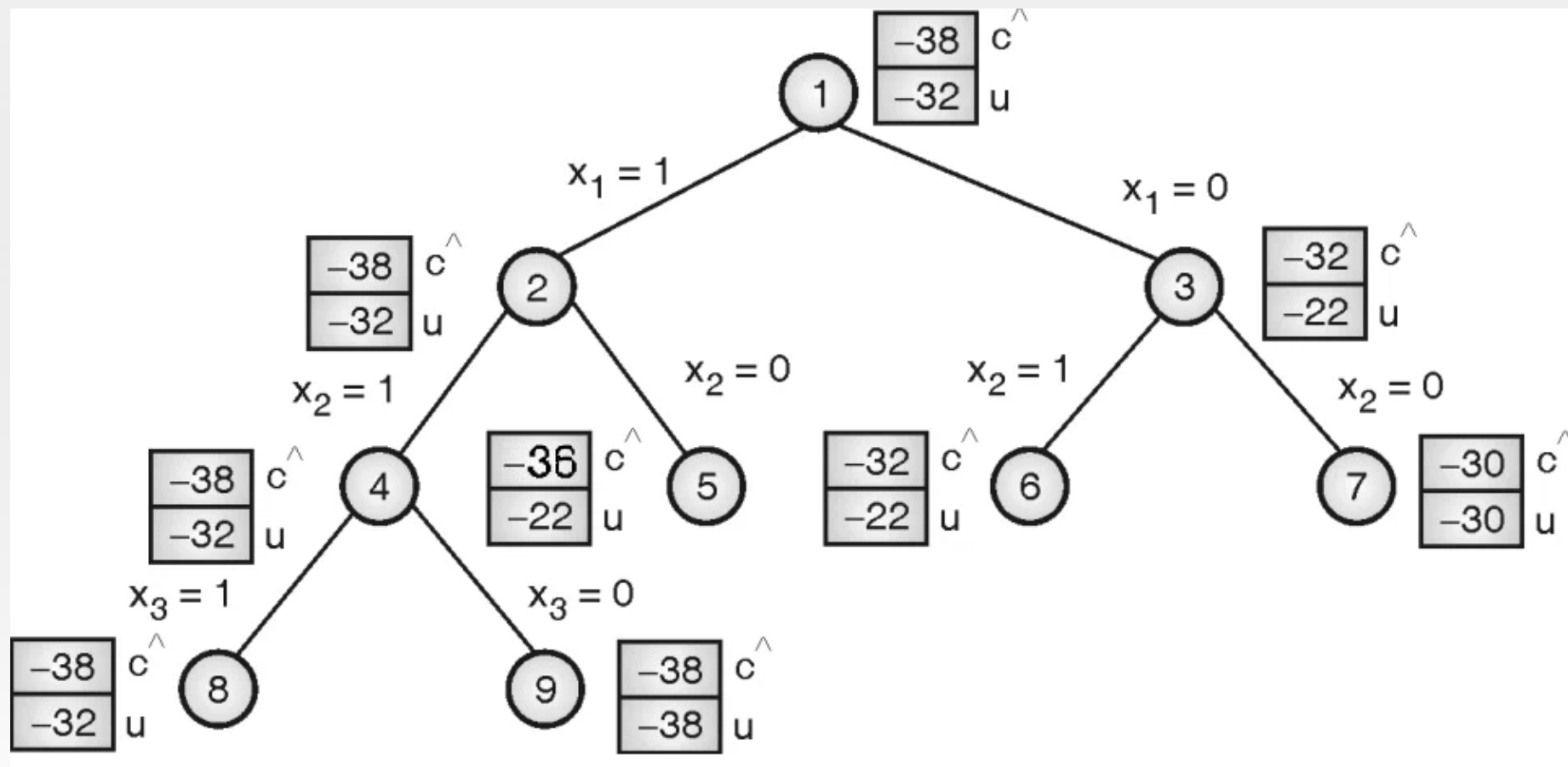
- $\hat{C}(9) = u(9) - (M - \text{Weight of selected items}) * \left(\frac{P_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(9) = -38 - (15 - (2 + 4 + 9)) * \left(\frac{0}{0}\right) = -38$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING FIFO - BB

State Space Tree:



i	P_i	W_i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 10: For $x_4 = 1$ – Item 4 selected for Node 8

- At node 8, item 4 cant be include the total weight will exceed the knapsack so Kill the node.

Node 11: For $x_4 = 0$ – Item 4 not selected for Node 8

- $u(11) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(11) = -(10 + 10 + 12) = -32$$

- $\hat{C}(11) = u(11) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(11) = -32 - (15 - (2 + 4 + 6)) * \left(\frac{0}{0}\right) = -32$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

0/1 KNAPSACK PROBLEM USING FIFO - BB

Node 12: For $x_4 = 1$ – Item 4 selected for Node 9

- $u(12) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(12) = -(10 + 10 + 18) = -38$$

- $\hat{C}(12) = u(12) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(12) = -38 - (15 - (2 + 4 + 9)) * \left(\frac{0}{0}\right) = -38$$

Node 13: For $x_4 = 0$ – Item 4 not selected for Node 9

- $u(13) = -\sum_i p_i$, such that $\sum_i w_i \leq M$,

$$u(13) = -(10 + 10) = -20$$

- $\hat{C}(13) = u(13) - (M - \text{Weight of selected items}) * \left(\frac{p_{i+1}}{w_{i+1}}\right)$

$$\hat{C}(13) = -20 - (15 - (2 + 4)) * \left(\frac{0}{0}\right) = -20$$

i	P _i	W _i
1	10	2
2	10	4
3	12	6
4	18	9

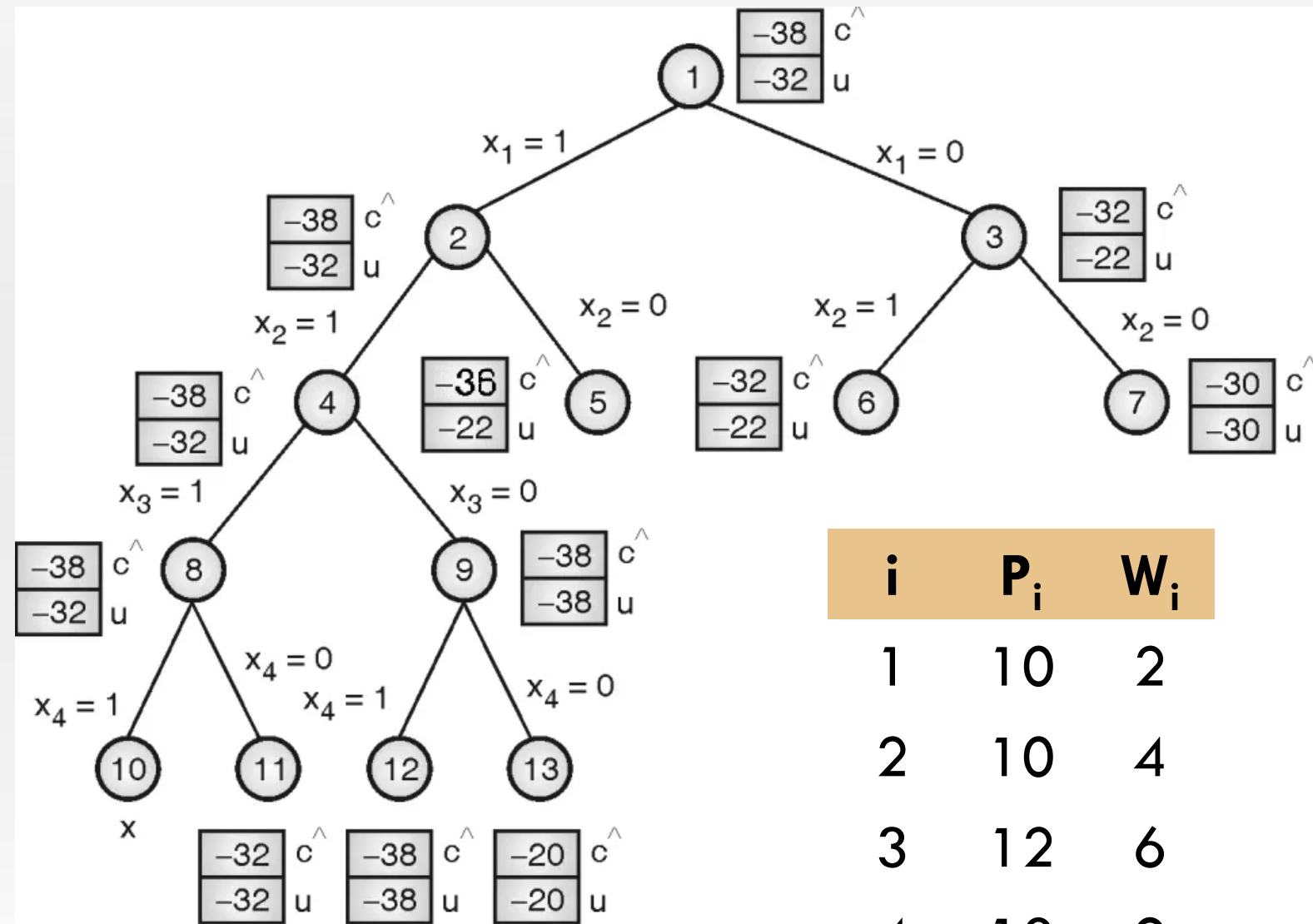
0/1 KNAPSACK PROBLEM USING FIFO - BB

State Space Tree:

Solution Vector

$$X = \{x_1, x_2, x_4\}$$

$$\text{Profit} = P_1 + P_2 + P_4 = 38$$



0/1 KNAPSACK PROBLEM USING BB

Algorithm:

BB_KNAPSACK(cp, cw, k)

BEGIN

cp $\leftarrow$ 0

cw $\leftarrow$ 0

k $\leftarrow$ 1

IF (cw + w[k] $\leq$ M) **THEN**

Y[k] $\leftarrow$ 1

IF (k < n) **THEN**

BB_KNAPSACK(cp + p[k], cw + w[k],
 k + 1)

END IF

IF (cp + p[k] > fp) && (k == n) **THEN**

 fp $\leftarrow$ cp + c[k]

 fw $\leftarrow$ cw + w[k]

 X $\leftarrow$ Y

END IF

END IF

IF **BOUND**(cp, cw, k) $\geq$ fp **THEN**

Y[k] $\leftarrow$ 0

IF (k < n) **THEN**

BB_KNAPSACK(cp, cw, k + 1)

END IF

IF (cp > fp) && (k == n) **THEN**

 fp $\leftarrow$ cp

 fw $\leftarrow$ cw

 X $\leftarrow$ Y

END IF

END IF

END FUNCTION

0/1 KNAPSACK PROBLEM USING BB

Algorithm:

BOUND(cp, cw, k)

BEGIN

$b \leftarrow cp$

$c \leftarrow cw$

FOR $i \leftarrow k + 1$ to n **DO**

IF $(c + w[i] \leq M)$ **THEN**

$c \leftarrow c + w[i]$

$b \leftarrow b - p[i]$

END

END

RETURN b

END FUNCTION

0/1 KNAPSACK PROBLEM USING BB

Time and Space Complexity

- Even though this method is more efficient than the other solutions to this problem, its worst case time complexity is still given by $O(2^n)$, in cases where the entire tree has to be explored.
- However, in its best case, only one path through the tree will have to be explored, and hence its best case time complexity is given by $O(n)$.
- Since this method requires the creation of the state space tree, its space complexity will also be exponential.



MODULE 2

BCSE204L – DESIGN AND
ANALYSIS OF ALGORITHM